МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении научно-исследовательской работы На тему: «Разработка имитатора пользовательских заявок и анализ эффективности распределения вычислительных ресурсов в планировщике SLURM»

Выполнил студент
группы 5130201/40002 _____ Храмов В.Д.

Принял старший преподаватель _____ Чуватов М.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

РЕФЕРАТ

Отчёт о научно-исследовательской работе изложен на 36 страницах, содержит 6 иллюстраций, 7 таблиц, 14 использованных источников, 1 приложение.

Ключевые слова: распределённые вычислительные системы, диспетчер заданий, SLURM, имитационное моделирование, нормальное распределение, логнормальное распределение, анализ датасета, виртуальный кластер, SSH.

Объектом исследования являются методы диспетчеризации и планирования вычислительных ресурсов в распределённых кластерных системах под управлением системы управления заданиями SLURM на базе операционной системы Linux в среде виртуализации.

Целью работы является разработка программного имитатора пользовательских заявок на основе статистического анализа исторических данных вычислительного кластера (вариант: UID 50430, JobName run_bwa.sh) и исследование динамики прохождения сформированных заявок через реальный диспетчер SLURM с применением механизмов масштабирования времени.

Методология работы включала развёртывание виртуального вычислительного кластера из пяти узлов средствами Oracle VM VirtualBox, статистический анализ выборки учётных данных acct_0921-0923 с вычислением границ диапазона фактического времени выполнения и параметров логарифма ошибки оценки времени выполнения, формируемой пользователем, а также натурный эксперимент по постановке сгенерированных заявок в очередь диспетчера.

Результатом разработки стал комплекс программно-управляемых скриптов на языках bash и Python, обеспечивающий фильтрацию исходного датасета, генерацию случайного потока заявок с усечением границ диапазона и их автоматическую отправку в пакетный диспетчер SLURM. Средствами SLURM собрана и проанализирована итоговая статистика соотношения «план — факт» для верификации использованной статистической модели.

Результаты работы могут быть использованы в учебном процессе при изучении технологий организации высокопроизводительных вычислений, администрирования распределённых систем под управлением SLURM.

Содержание

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 4

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 7

ВВЕДЕНИЕ 8

1 ОСНОВНАЯ ЧАСТЬ 10

1.1 Постановка задачи 10

1.2 Организация виртуальной вычислительной среды 12

1.3 Разработка имитатора пользовательских заявок 16

1.4 Результаты экспериментов и анализ данных 23

ЗАКЛЮЧЕНИЕ 29

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 31

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД 33

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями:

1. Виртуализация — технология создания программного (виртуального) представления вычислительных ресурсов (процессора, памяти, накопителей, сетевых интерфейсов), позволяющая запускать несколько независимых операционных систем на одном физическом устройстве.
2. Виртуальная машина (ВМ) — программная эмуляция вычислительного устройства, предоставляющая гостевой операционной системе доступ к выделенным ей виртуальным аппаратным ресурсам посредством виртуализации.
3. VirtualBox (Oracle VM VirtualBox) — программный продукт виртуализации для операционных систем Windows, Linux, macOS и других, позволяющий запускать гостевые операционные системы в изолированной среде виртуальных машин на базе хост-системы.
4. Вычислительный кластер — совокупность взаимосвязанных вычислительных узлов, объединённых высокоскоростной сетью и функционирующих как единая вычислительная система под управлением специализированного программного обеспечения.
5. IP-адрес (Internet Protocol address) — уникальный числовой идентификатор сетевого интерфейса узла в компьютерной сети, используемый для адресации и маршрутизации пакетов данных по протоколу IP.
6. TCP-соединение (Transmission Control Protocol) — логическое соединение между двумя узлами сети, устанавливаемое по протоколу TCP и обеспечивающее надёжную, упорядоченную и контролируемую на предмет ошибок передачу данных между ними.
7. SSH (Secure Shell) — сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP-соединений, в том числе с использованием авторизации по криптографическим ключам без передачи пароля по сети.
8. DNS (Domain Name System) — распределённая система преобразования символьных доменных имён узлов сети в соответствующие им

IP-адреса и обратно.

9. Разрешение имён — процесс сопоставления символьных имён узлов сети соответствующим им IP-адресам, выполняемый средствами DNS или локального файла соответствий (/etc/hosts).
10. Диспетчер заданий — программный компонент системы управления кластером, отвечающий за приём заявок от пользователей, ведение очереди заданий, планирование и распределение ресурсов вычислительных узлов.
11. Управляющий узел (head node, control node) — узел вычислительного кластера, на котором выполняется диспетчер заданий и средства управления кластером, отвечающие за приём, планирование и распределение заданий по вычислительным узлам.
12. Вычислительный узел (compute node) — узел вычислительного кластера, предоставляющий ресурсы (процессорное время, память) для непосредственного выполнения заданий, поставленных в очередь диспетчера.
13. Munge — служба аутентификации, используемая для проверки подлинности сообщений, которыми обмениваются узлы кластера, на основе общего секретного ключа.
14. SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера с открытым исходным кодом, обеспечивающая постановку заданий в очередь, их планирование и распределение по вычислительным узлам; для аутентификации узлов использует службу Munge.
15. Заявка (задание, job) — описание вычислительной задачи, поставленной пользователем в очередь диспетчера кластера, включающее запрошенные ресурсы и оценку времени выполнения.
16. Очередь заданий диспетчера — упорядоченный список заявок, принятых диспетчером SLURM и ожидающих выделения ресурсов для запуска на вычислительных узлах.
17. Среднее арифметическое — статистическая характеристика выборки, равная сумме всех её значений, делённой на их количество.
18. Дисперсия — статистическая характеристика разброса значений случайной величины, равная среднему арифметическому квадратов от-

клонений значений от среднего арифметического выборки.

19. Стандартное отклонение — статистическая характеристика разброса значений случайной величины относительно её среднего арифметического, равная квадратному корню из дисперсии выборки.
20. Нормальное распределение — непрерывное распределение вероятностей случайной величины, полностью определяемое математическим ожиданием и стандартным отклонением; график плотности имеет симметричную колоколообразную форму.
21. Логнормальное распределение — распределение вероятностей случайной величины, логарифм которой имеет нормальное распределение с параметрами μ и σ.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

ВМ — виртуальная машина

ОЗУ — оперативное запоминающее устройство

ОС — операционная система

ПО — программное обеспечение

ЦП — центральный процессор

CPU (Central Processing Unit) — центральный процессор

DNS (Domain Name System) — система доменных имён

GID (Group Identifier) — идентификатор группы пользователей

HPC (High Performance Computing) — высокопроизводительные вычисления

IP (Internet Protocol) — межсетевой протокол

Munge (MUNGE Uid 'N' Gid Emporium) — служба аутентификации узлов кластера на основе общего секретного ключа

NAT (Network Address Translation) — преобразование сетевых адресов

RAM (Random Access Memory) — оперативная память

SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера

SSH (Secure Shell) — сетевой протокол прикладного уровня для удалённого управления операционной системой

TCP (Transmission Control Protocol) — протокол управления передачей

UID (User Identifier) — идентификатор пользователя

ВВЕДЕНИЕ

В современном мире, где объёмы обрабатываемых данных постоянно растут, а вычислительные задачи становятся всё сложнее, технологии высокопроизводительных вычислений (HPC) приобретают критически важное значение [2; 11]. Их применение позволяет объединять ресурсы множества вычислительных узлов в единый кластер, обеспечивая тем самым существенное увеличение производительности и сокращение времени решения сложных научных и инженерных задач.

Эффективная эксплуатация вычислительного кластера невозможна без системы управления заданиями и ресурсами, обеспечивающей приём заявок пользователей, их постановку в очередь и распределение по вычислительным узлам в соответствии с заданной политикой планирования. Одной из наиболее распространённых систем такого класса является SLURM [8; 13], широко используемая как в коммерческих, так и в научно-исследовательских вычислительных центрах.

Освоение принципов администрирования и эксплуатации вычислительного кластера на базе SLURM не требует обязательного наличия физического кластера: для учебных и исследовательских целей вычислительная среда может быть воспроизведена в виде набора виртуальных машин. Такие инструменты, как VirtualBox [6], позволяют создавать изолированные виртуальные среды, в которых каждая машина выполняет роль отдельного узла кластера, что делает возможным изучение архитектуры распределённой системы средствами одного физического компьютера.

Операционные системы семейства Linux, в частности дистрибутив Fedora Server [1], благодаря своей гибкости, открытости и широкому набору системных утилит, предоставляют все необходимые инструменты для организации управляющего и вычислительных узлов кластера в режиме интерфейса командной строки. Для организации безопасного взаимодействия между узлами кластера и удалённого администрирования незаменим протокол SSH, обеспечивающий авторизацию по криптографическим ключам без передачи пароля по сети, а также служба munge, отвечающая за аутентификацию сообщений между демонами SLURM.

Помимо развёртывания и настройки самой вычислительной среды, важной задачей является имитация реальной пользовательской нагрузки

на кластер с целью проверки корректности работы диспетчера и последующего анализа эффективности планирования заданий. Для построения такого имитатора целесообразно опираться не на произвольные, а на статистически обоснованные параметры заявок, полученные путём анализа реальных данных учёта выполненных задач действующего вычислительного кластера. Применение математического аппарата теории вероятностей и математической статистики, в частности нормального и логнормального распределений [14], позволяет генерировать синтетические заявки, по своим временным характеристикам приближенные к наблюдаемым в реальных системах.

В ходе настоящей НИР рассматриваются технологии организации вычислительного кластера на базе виртуализации и Linux, включая развёртывание управляющего и вычислительных узлов средствами VirtualBox, настройку сетевого взаимодействия и SSH-доступа, установку и конфигурирование диспетчера SLURM, а также разработку имитатора пользовательских заявок, основанного на статистическом анализе набора учётных данных кластера.

1 ОСНОВНАЯ ЧАСТЬ

1.1 Постановка задачи

В соответствии с выданным заданием (вариант: UID - 50430, JobName - run_bwa.sh) в рамках научно-исследовательской работы необходимо решить комплекс задач, разделённый на три основных этапа:

1. Подготовка виртуальной среды кластера. Необходимо развернуть и настроить виртуальную среду, имитирующую инфраструктуру распределённых вычислительных ресурсов суперкомпьютера, со следующими техническими характеристиками и требованиями:
   (a) Развернуть 5 виртуальных машин в среде виртуализации Oracle VM VirtualBox: один управляющий узел (master) с диспетчером ресурсов SLURM и средствами разработки, а также 4 вычислительных узла (node1–node4).
   (b) Выделить под каждый узел конфигурацию в пределах 1–2 виртуальных ЦП и 2–4 ГБ оперативной памяти. На всех машинах должна функционировать операционная система Linux в режиме интерфейса командной строки (CLI) без графической оболочки.
   (c) Настроить изолированную виртуальную сеть между всеми узлами с поддержкой разрешения имён.
   (d) Обеспечить беспарольный доступ по протоколу SSH между управляющим и вычислительными узлами с авторизацией по ключам.
   (e) Выполнить установку, настройку и запуск распределённого пакетного диспетчера ресурсов SLURM. Ресурсной единицей планирования определить вычислительный узел; другие типы ресурсов (ядра, оперативная память, дисковые квоты) в рамках данной работы не учитываются.
   (f) Все действия в рамках экспериментов должны выполняться под единой для всех узлов непривилегированной учётной записью пользователя (vlad). Суперпользователь root применяется исключительно для конфигурирования системного ПО.
2. Разработка имитатора пользовательских заявок. Программный ком-

плекс имитации должен реализовывать статистическую модель генерации нагрузки на основе исторических данных и включать в себя:

(a) Предварительный статистический анализ исторического набора данных acct_0921-0923. Требуется отфильтровать записи, соответствующие индивидуальному варианту, и вычислить:
- Границы диапазона фактического времени выполнения задач ($T_{\min}$ и $T_{\max}$);
- Параметры логнормального распределения ошибки оценки времени: среднее арифметическое логарифмов ошибки (μ) и их стандартное отклонение (σ).

(b) Реализацию фазы масштабирования времени с применением понижающих коэффициентов к временным интервалам ($T_{\min}, T_{\max}$) для возможности проведения натурного эксперимента в разумные сроки без искажения математической логики модели.

(c) Использование относительной величины ошибки оценки времени $\tau = T_{\text{заявки}}/T_{\text{факт}}$ (ввиду соотношения $\Delta T > T_{\text{факт}}$ для текущего варианта) для расчёта планового времени.

(d) Алгоритм генерации отдельной заявки, где:
- Фактическое время выполнения (t_{fact}) моделируется как случайная величина в усечённом диапазоне $[T_{\min}, T_{\max}]$ по нормальному закону распределения;
- Запрашиваемое пользователем время (t_{req}) вычисляется на основе логнормального отклонения τ от сгенерированного значения t_{fact};
- Потребность в вычислительных узлах выбирается из дискретного множества $\{1, 2, 3, 4\}$ на основе равномерного распределения.

(e) Контроль входных параметров: имитатор должен принимать на вход объём выборки N (целое положительное число, $1 \leq N \leq 999$).

(f) Автоматическую и последовательную отправку сформированных заявок в реальную очередь диспетчера SLURM.

3. Сбор статистики и верификация эксперимента. Для оценки адекватности работы имитатора необходимо организовать систему объектив-

ного контроля:

(a) Реализовать автоматизированный сбор данных непосредственно в процессе прохождения задач через диспетчер ресурсов (метрика «план – факт»).

(b) Сохранять в независимые файлы логов как исходные сгенерированные параметры («план» имитатора), так и фактические системные метрики времени выполнения и терминальных состояний процессов, зафиксированные средствами SLURM («факт» планировщика).

1.2 Организация виртуальной вычислительной среды

Выбор и развёртывание среды виртуализации

В качестве среды виртуализации в данной работе использовался программный продукт Oracle VM VirtualBox [6], выбор которого обусловлен его свободным распространением, поддержкой широкого спектра гостевых операционных систем и наличием официальной сборки для архитектуры ARM64, что является необходимым условием для развёртывания виртуальной среды на хост-системе с процессором Apple Silicon (MacBook Air, чип Apple M2, 8 ГБ ОЗУ).

В соответствии с вариантом задания в качестве гостевой операционной системы был выбран дистрибутив Fedora Server [1] в редакции для архитектуры aarch64. Установка и функционирование системы осуществлялись в режиме интерфейса командной строки, без графической оболочки.

Архитектура виртуального кластера

Для имитации вычислительных ресурсов суперкомпьютера было развёрнуто пять виртуальных машин: один управляющий узел (master), на котором размещаются диспетчер SLURM и средства разработки, и четыре вычислительных узла (node1-node4), предоставляющие ресурсы для выполнения заданий. Конфигурация виртуальных машин, выбранная с учётом ограничений хост-системы по объёму оперативной памяти, приведена в таблице 1.

Таблица 1: Конфигурация узлов виртуального кластера

Узел	Роль	ЦП (ядра)	ОЗУ (МБ)
master	управляющий узел	2	2048
node1	вычислительный узел	1	1024
node2	вычислительный узел	1	1024
node3	вычислительный узел	1	1024
node4	вычислительный узел	1	1024

На каждом узле создана единая непривилегированная учётная запись пользователя(vlad), используемая для запуска заданий и работы с диспетчером SLURM; учётная запись root применялась исключительно для установки и настройки системного программного обеспечения.

Настройка виртуальной сети и разрешения имён

Для обеспечения сетевого взаимодействия внутри кластера на каждой виртуальной машине был настроен сетевой адаптер в режиме внутренней сети (Internal Network), формирующий изолированную локальную подсеть 192.168.1.0/24. Для обеспечения доступа с хост-машины к управляющему узлу дополнительно задействован интерфейс NAT с пробросом порта 2222 хоста на порт 22 гостевой ОС. Каждому узлу присвоен статический IP-адрес согласно таблице 2.

Таблица 2: Адресация узлов виртуального кластера

Имя узла	IP-адрес
master	192.168.1.10
node1	192.168.1.11
node2	192.168.1.12
node3	192.168.1.13
node4	192.168.1.14

Разрешение имён узлов реализовано посредством редактирования файла /etc/hosts на каждой виртуальной машине. Данный подход позволяет обращаться к узлам по символьным именам (master, node1–node4) без развёртывания внешнего DNS-сервера и обеспечивает корректную работу демонов планировщика SLURM [8]. Итоговое содержимое файла приведено в листинге 1.

```
1 127.0.0.1     localhost
```

```
192.168.1.10 master
192.168.1.11 node1
192.168.1.12 node2
192.168.1.13 node3
192.168.1.14 node4
```

Листинг 1: Содержимое файла /etc/hosts

Настройка SSH с авторизацией по ключам

На управляющем узле для непривилегированной учётной записи сгенерирована пара криптографических ключей — приватный и публичный — с использованием утилиты ssh-keygen (листинг 2). Публичный ключ распространён на все вычислительные узлы посредством утилиты ssh-copy-id, что обеспечивает выполнение удалённых подключений и команд без запроса пароля.

```
ssh-keygen -t ed25519 -C "cluster-key"
ssh-copy-id vlad@node1
ssh-copy-id vlad@node2
ssh-copy-id vlad@node3
ssh-copy-id vlad@node4
```

Листинг 2: Генерация и распространение ключей SSH

В конфигурационном файле демона SSH (/etc/ssh/sshd_config) на всех узлах отключена авторизация по паролю посредством установки параметра PasswordAuthentication no, что исключает возможность входа без криптографического ключа.

Установка и настройка munge

Для аутентификации сообщений, которыми обмениваются демоны SLURM на разных узлах, на управляющем узле был сгенерирован общий секретный ключ munge.key, который был скопирован на все вычислительные узлы с сохранением единых владельца и прав доступа (пользователь munge). Служба munge была запущена и добавлена в автозагрузку на всех пяти узлах кластера.

Установка и настройка SLURM

На управляющем узле был установлен и настроен демон диспетчера slurmctld, на вычислительных узлах — демон slurmd. Основные параметры кластера определены в общем для всех узлов конфигурационном файле /etc/slurm/slurm.conf: имя кластера, адрес управляющего узла, список вычислительных узлов с указанием количества процессоров и объёма оперативной памяти, а также определение единственного раздела (partition), включающего все четыре вычислительных узла.

```
ClusterName=mac_cluster
SlurmctldHost=master
SlurmUser=slurm
SlurmdUser=root
AuthType=auth/munge
CryptoType=crypto/munge
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
SlurmdParameters=config_overrides

# Логи
SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
StateSaveLocation=/var/spool/slurmctld

# Описание машин
NodeName=node[1-4] CPUs=1 State=UNKNOWN
PartitionName=debug Nodes=node[1-4] Default=YES MaxTime=INFINITE State=UP
```

Листинг 3: Фрагмент файла slurm.conf

После запуска демонов на всех узлах корректность конфигурации и доступность вычислительных узлов проверялась командами sinfo и scontrol show nodes.

```
vlad@master:~$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*       up   infinite      4   idle node[1-4]
```

Листинг 4: Вывод команды sinfo

```
vlad@master:~$ scontrol show nodes
NodeName=node1 Arch=aarch64 CoresPerSocket=1
   CPUAlloc=0 CPUEfctv=1 CPUTot=1 CPULoad=0.18
   AvailableFeatures=(null)
```

```
   ActiveFeatures=(null)
   Gres=(null)
   NodeAddr=node1 NodeHostName=node1 Version=24.05.2
   OS=Linux 6.19.10-300.fc44.aarch64 #1 SMP PREEMPT_DYNAMIC Wed Mar 25
    17:45:07 UTC 2026
   RealMemory=1 AllocMem=0 FreeMem=463 Sockets=1 Boards=1
   State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
   Partitions=debug
   BootTime=2026-06-25T06:44:34 SlurmdStartTime=2026-06-25T06:44:45
   LastBusyTime=2026-06-25T06:44:44 ResumeAfterTime=None
   CfgTRES=cpu=1,mem=1M,billing=1
   AllocTRES=
   CurrentWatts=0 AveWatts=0
```

Листинг 5: Фрагмент вывода команды scontrol show nodes

Клонирование вычислительных узлов

После полной настройки и проверки работоспособности первого вычислительного узла (установка ОС, сеть, SSH, munge, slurmd) данная виртуальная машина использовалась как базовый образ для клонирования оставшихся четырех вычислительных узлов средствами VirtualBox.

1.3 Разработка имитатора пользовательских заявок

Анализ исходного набора данных

Исходные данные для построения статистической модели пользовательских заявок представлены в виде набора учётных записей SLURM acct_0921-0923 [9], содержащего сведения о реально выполненных задачах действующего вычислительного кластера. В соответствии с вариантом задания из общего набора данных были отобраны записи, удовлетворяющие условиям фильтрации: идентификатор пользователя UID = 50430 и имя задачи JobName = "run_bwa.sh".

В результате первичной фильтрации было получено $N_{\text{sample}} = 1433$ записи, послужившие основой для дальнейшего статистического анализа.

В ходе предварительного анализа отобранного подмножества выявлено 80 записей со статусом COMPLETED и фактическим временем выполнения не более 10 секунд. Данные задачи относятся к одному инфраструктурному событию 30 августа 2023 года и характеризуются аномально высоким коэффициентом ошибки оценки

$\tau = T_{\text{заявки}}/T_{\text{факт}}$ в диапазоне от 7200 до 43200, что на два порядка превышает типичные значения для нормальных задач ($\tau \approx 10$–200) [4; 5]. Природа данной аномалии связана со сбоем вычислительного узла (NODE_FAIL) [3], после которого планировщик зафиксировал аварийно завершенные и автоматически перезапущенные задачи как успешно выполненные с околонулевым временем работы. Указанные записи были исключены из расчёта параметров модели как артефакты инфраструктуры, не характеризующие реальную пользовательскую нагрузку. Итоговая очищенная выборка для статистического анализа составила 703 записи.

Предварительный анализ и масштабирование данных

Анализ распределения параметров показал, что фактическое время выполнения пользовательских задач (ElapsedRaw) варьируется в широком диапазоне от 29 секунд до 10794 секунд (около 3 часов).

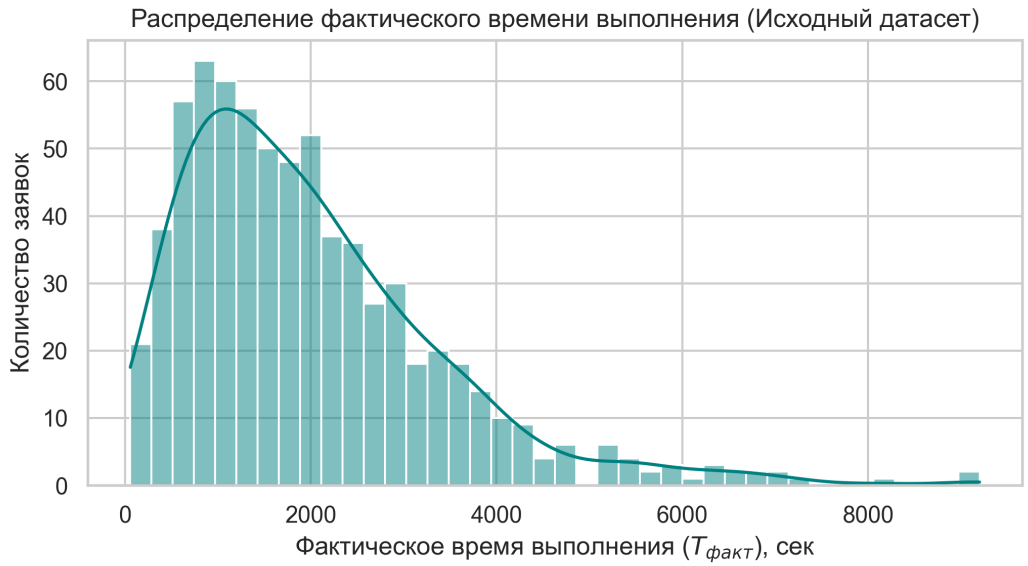


Рис. 1: Распределение фактического времени выполнения задач (исходный датасет acct_0921-0923)

Статистический анализ логарифмированных значений коэффициента ошибки пользовательской оценки ($\ln \tau$) по 703 записям дал следующие параметры распределения: математическое ожидание $\mu = 3{,}9411$ и среднеквадричное отклонение $\sigma = 0{,}8260$. Данный график распределения подтверждает применимость логнормальной модели для описания ошибки пользовательской оценки времени выполнения задач данного типа [4; 12].

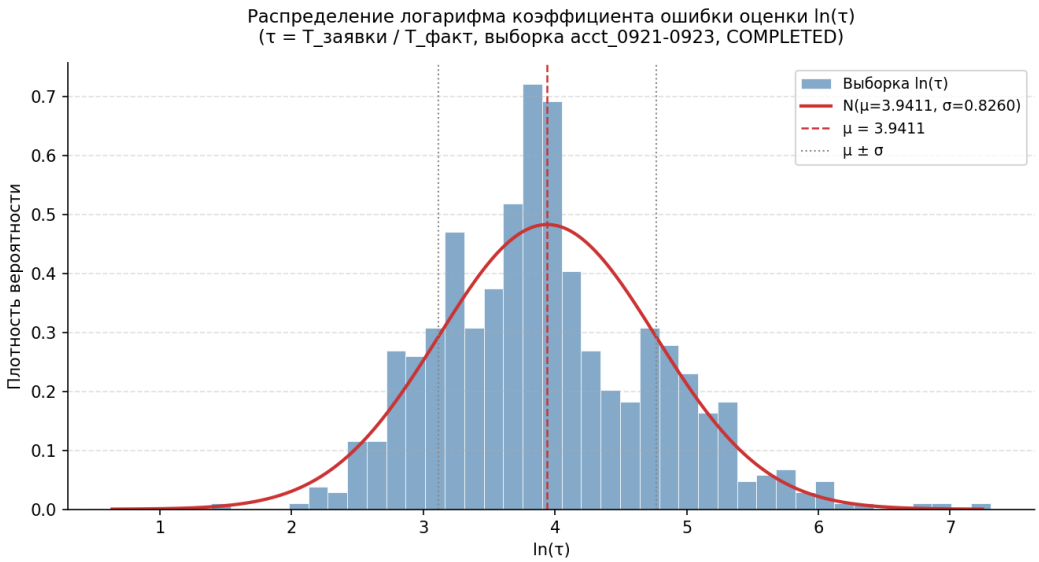


Рис. 2: Распределение логарифма коэффициента ошибки оценки $\ln(\tau)$ на основе исторических данных

Широкий временной диапазон исходных задач существенно затрудняет их непосредственное воспроизведение на учебном демонстрационном эксперименте. В связи с этим в процессе разработки было принято решение о временном масштабировании характеристик. Для финального тестирования и валидации модели выбран понижающий коэффициент масштабирования $k = 30$. Данное преобразование переводит диапазон выполнения основной массы заявок в интервалы от единиц секунд до единиц минут (максимальное масштабированное время составило 360 секунд, а медианное значение — около 30 секунд). Такой масштаб минимизирует интегральную ошибку округления диспетчера, сокращает общее время проведения эксперимента и сохраняет достаточную долю задач длительностью свыше одной минуты ($\approx$18% выборки), что критически важно для корректного моделирования сценариев принудительного снятия задач планировщиком по таймауту.

Для снижения общей трудоемкости и ускорения валидации имитационной модели без потери статистической значимости, объем финальной тестовой выборки был установлен в размере примерно 0,5 от величины исходного набора данных и составил $N_{\text{sim}} = 694$ заявки.

Выбор модели ошибки оценки времени выполнения

Для количественной характеристики отклонения пользовательской оценки времени выполнения от фактической длительности задачи использовались две альтернативные модели:

- абсолютное отклонение: $\Delta T = T_{\text{заявки}} - T_{\text{факт}}$;
- относительный коэффициент ошибки: $\tau = T_{\text{заявки}}/T_{\text{факт}}$.

По результатам анализа было установлено, что дисперсия величины ΔT сильно зависит от масштаба самой задачи и часто превышает значение $T_{\text{факт}}$, в то время как относительный коэффициент τ инвариантен к длительности задачи. На основании этого для имитатора была выбрана логнормальная модель относительного отклонения τ [5; 12].

Вычисление статистических параметров

В таблице 3 приведены итоговые сопоставимые параметры исходного очищенного набора данных и результирующей масштабированной выборки, используемой в имитаторе.

Таблица 3: Статистические параметры исходной и масштабированной выборок

Параметр	Исходное значение	Масштабированное значение ($k = 30$)
$T_{\text{факт, min}}$, с	29	1
$T_{\text{факт, max}}$, с	10794	360
μ ($\ln \tau$)	3,9411	3,9411 (не изменяется)
σ ($\ln \tau$)	0,8260	0,8260 (не изменяется)

Алгоритм генерации пользовательской заявки

Программная генерация параметров каждой синтетической заявки реализована на языке Python [7] и состоит из трех последовательных шагов.

На первом этапе моделируется фактическое время выполнения задачи $T_{\text{факт}}$ как случайная величина, подчиненная нормальному закону распределения $\mathcal{N}(\mu_t, \sigma_t^2)$. Математическое ожидание и стандартное отклонение определяются на основе масштабированных границ исследуемой выборки:

$$\mu_t = \frac{T_{\min} + T_{\max}}{2} = 180{,}5 \text{ с}, \qquad \sigma_t = \frac{T_{\max} - T_{\min}}{4} = 89{,}75 \text{ с}.$$

Данный выбор обеспечивает попадание более 95% генерируемых значений в интервал $[T_{\min}, T_{\max}]$. Ограничение диапазона сверху и снизу реализуется методом усечения (генерация в цикле до выполнения условия $1 \leq T_{\text{факт}} \leq 360$). В программном коде также предусмотрено жесткое ограничение:

$$T_{\text{факт}} = \max(2, \text{round}(T_{\text{факт}})) \tag{1}$$

Введение нижней граничной константы в 2 секунды технически обосновано необходимостью исключить генерацию сверхкоротких задач (1 с), для которых накладные расходы на вызов системного таймера и инициализацию интерпретатора сопоставимы с временем работы самой задачи, что приводит к значительным погрешностям при округлении интервалов.

На втором этапе моделируется коэффициент пользовательской ошибки перестраховки $\tau \sim \text{LogNormal}(\mu, \sigma^2)$ с параметрами $\mu = 3{,}9411$ и $\sigma = 0{,}8260$. Плановое запрашиваемое время вычисляется как:

$$T_{\text{заявки}} = \text{round}(T_{\text{факт}} \cdot \tau)$$

Если сгенерированное значение удовлетворяет неравенству $T_{\text{заявки}} \leq T_{\text{факт}}$, лимит времени принудительно корректируется добавлением случайной величины в диапазоне $[5, 15]$ секунд, что гарантирует штатное выполнение задачи.

На третьем этапе учитывается специфика работы пакетного диспетчера SLURM: директива лимита времени –time в формате MM:SS принудительно округляется планировщиком вверх до ближайшей целой минуты. Следовательно, при сильном масштабировании короткие задачи гарантированно успевают завершиться в рамках выделенного минутного кванта, из-за чего механизм принудительного снятия по таймауту (TIMEOUT) диспетчером зафиксирован не будет. Для обеспечения возможности полноценного тестирования и валидации подсистемы обработки прерываний, вероятность генерации таймаутных заявок искусственно увеличена в рамках эксперимента. Принудительный таймаут моделируется только для задач с $T_{\text{факт}} > 60$ с, при этом плановый лимит времени $T_{\text{заявки}}$ устанавливается строго краттным 60 секундам и гарантированно меньшим, чем реальное время выполнения задачи $T_{\text{факт}}$.

Количество запрашиваемых задачей вычислительных узлов моделируется как дискретная случайная величина, распределенная равномерно на множестве $\{1, 2, 3, 4\}$.

Реализация программы-имитатора и структура данных

Программа-имитатор реализована в виде скрипта simulator.py. Перед запуском цикла генерации скрипт производит очистку целевого каталога /home/vlad/saved_jobs и открывает на перезапись файлы логирования. В процессе работы формируются три типа файлов данных, структуры которых подробно описаны в таблицах 4, 5 и 6.

Таблица 4: Структура файла внутренней статистики simulation_stats.csv

Поле	Тип	Описание
Timestamp	строка	Дата и время генерации параметров задачи (ГГГГ-ММ-ДД ЧЧ:ММ:СС)
SimID	целое	Внутренний порядковый номер заявки в имитаторе
SlurmJobID	целое	Идентификатор, присвоенный задаче планировщиком SLURM
Nodes	целое	Количество запрошенных вычислительных узлов (1–4)
PlannedTime_Sec	целое	Сгенерированный плановый лимит времени $T_{\text{заявки}}$, с
ActualTime_Sec	целое	Смоделированное базовое фактическое время выполнения $T_{\text{факт}}$, с
Tau	вещест.	Реализованный коэффициент ошибки оценки $\tau = T_{\text{заявки}}/T_{\text{факт}}$
IsTimeout	бинарн.	Флаг отнесения заявки к категории таймаутных (0 — нет, 1 — да)

Таблица 5: Структура системного лога планировщика slurm_real_fact.txt

Поле	Тип	Описание
JobID	целое	Уникальный идентификатор задачи в подсистеме учета SLURM
JobName	строка	Имя запущенного пакетного скрипта (run_bwa.sh)
Nodes	целое	Фактически выделенное количество вычислительных узлов
State	строка	Терминальный статус задачи (COMPLETED, TIMEOUT)
ElapsedRaw	целое	Фактическое время выполнения задачи по часам SLURM, с

Таблица 6: Структура лога высокоточного таймера overhead_log.csv

Поле	Тип	Описание
SlurmJobID	целое	Идентификатор задача в планировщике
SleepRequested_Sec	целое	Длительность вычислительной нагрузки, переданная в команду sleep, с
ActualElapsed_Ms	целое	Полное время выполнения обертки задачи, измеренное утилитой date с точностью до миллисекунды, мс

Каждая сгенерированная задача экспортируется в текстовый файл сценария sbatch [10], содержащий директивы выделения ресурсов. Для проведения детального исследования накладных расходов диспетчеризации в тело каждого сценария встроен независимый замер времени аппаратными средствами ОС до и после выполнения полезной нагрузки с точностью до миллисекунды (с использованием системного вызова date +%s%3N):

START=$(date +%s%3N)
sleep <T_fact>
END=$(date +%s%3N)

Разность этих меток фиксируется в файле overhead_log.csv, что позволяет оценить системные задержки, которые SLURM не учитывает в своем стандартном логе, ведущемся дискретно до секунд.

Ниже приведены сопоставимые фрагменты полученных файлов данных для первых пяти задач симуляционного эксперимента.

```
Timestamp,SimID,SlurmJobID,Nodes,PlannedTime_Sec,ActualTime_Sec,Tau,IsTimeout
2026-06-29 15:46:43,1,1294,3,16208,316,51.2927,0
2026-06-29 15:46:50,2,1295,4,8355,254,32.8953,0
2026-06-29 15:47:02,3,1296,1,23453,305,76.8959,0
2026-06-29 15:47:16,4,1297,4,27753,208,133.4287,0
2026-06-29 15:47:24,5,1298,3,22791,246,92.6476,0
```

Листинг 6: Фрагмент файла внутренней статистики имитатора simulation_stats.csv

```
JobID|JobName|Nodes|State|ElapsedRaw
1294|run_bwa.sh|3|COMPLETED|316
1295|run_bwa.sh|4|COMPLETED|254
1296|run_bwa.sh|1|COMPLETED|305
1297|run_bwa.sh|4|COMPLETED|208
1298|run_bwa.sh|3|COMPLETED|246
```

Листинг 7: Фрагмент системного лога планировщика slurm_real_fact.txt

```
SlurmJobID,SleepRequested_Sec,ActualElapsed_Ms
1294,316,316048
1295,254,254103
1296,305,305017
1297,208,208060
1298,246,246104
```

Листинг 8: Фрагмент лога накладных расходов overhead_log.csv

Сопоставление полей ActualTime_Sec из лога имитатора и ElapsedRaw из лога планировщика наглядно подтверждает точность воспроизведения временных параметров в состоянии COMPLETED. Анализ данных файла overhead_log.csv показывает наличие систематических накладных расходов диспетчеризации (величина задержки инициализации лежит в диапазоне от 17 до 104 мс), детальное исследование которых приведено в следующем аналитическом разделе работы.

1.4 Результаты экспериментов и анализ данных

Условия проведения эксперимента

В рамках экспериментальной проверки работы разработанного программного комплекса было сгенерировано и последовательно поставлено в очередь диспетчера $N = 716$ синтетических заявок (что составляет примерно 0,5 от объёма исходного набора данных для снижения временных затрат и валидации модели). Все четыре вычислительных узла кластера (node1–node4) на момент начала эксперимента находились в состоянии полной готовности (idle), управляющий узел master успешно координировал распределение ресурсов в изолированной сети.

Результаты постановки заявок в очередь

По завершении выполнения всего пула сгенерированных задач из базы данных подсистемы учёта SLURM (посредством утилиты sacct [9]) и внутренних логов имитатора были собраны итоговые статистические характеристики. Сводные результаты сопоставления плановых параметров и фактических метрик выполнения для первых пяти заявок эксперимента

представлены в таблице 7.

Таблица 7: Сравнение плановых и фактических характеристик выполнения первых заявок

SlurmJobID	$T_{\text{заявки}}$ (план), с	$T_{\text{факт}}$ (имитатор), с	Узлов	Состояние
1294	16208	316	3	COMPLETED
1295	8355	254	4	COMPLETED
1296	23453	305	1	COMPLETED
1297	27753	208	4	COMPLETED
1298	22791	246	3	COMPLETED

Анализ отклонений «план – факт»

Сопоставление запрошенного пользователем лимита времени $T_{\text{заявки}}$ и фактического времени выполнения задач выполнено по результатам симуляционного эксперимента для всей выборки из $N = 716$ записей.

Из общего числа сгенерированных заявок 58 задач (8,10 %) были классифицированы моделью как таймаутные. Для данных задач плановый лимит времени устанавливался строго меньше фактического и квантовался с округлением вверх до ближайшей минуты в соответствии с системной логикой SLURM. Анализ файла системного аудита slurm_real_fact.txt показал, что все 58 заявок были принудительно остановлены диспетчером и получили терминальный статус TIMEOUT, при этом зафиксированное планировщиком время ElapsedRaw строго соответствовало назначенному лимиту, что подтверждает корректность настройки подсистемы прерываний кластера.

Статистическая валидация параметров распределения

Для оставшихся 658 нормальных заявок со статусом COMPLETED по значениям $T_{\text{заявки}}$ и $T_{\text{факт}}$ были вычислены выборочные логарифмы коэффициента ошибки пользователя $\ln \tau = \ln(T_{\text{заявки}}/T_{\text{факт}})$. На их основе методом максимального правдоподобия получены эмпирические оценки параметров логнормального распределения:

$$\hat{\mu} = 3{,}9295, \qquad \hat{\sigma} = 0{,}8151. \tag{2}$$

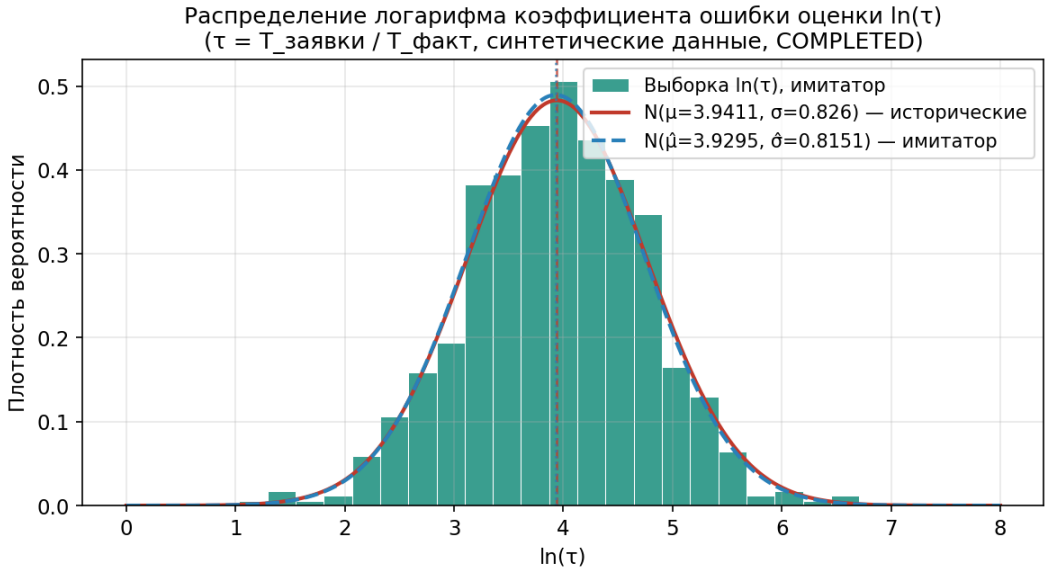


Рис. 3: Распределение логарифма коэффициента ошибки $\ln(\tau)$ по синтетическим данным

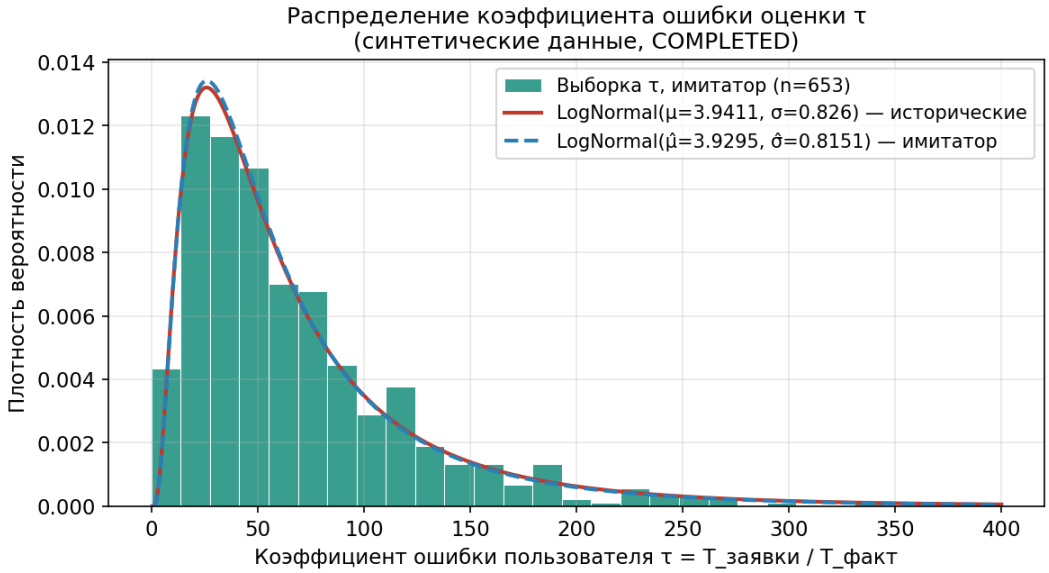


Рис. 4: Гистограмма коэффициента ошибки пользователя τ (синтетические данные)

Полученные эмпирические значения демонстрируют высокую степень близости к теоретическим параметрам, заложенным в имитатор при генерации ($\mu = 3{,}9411$, $\sigma = 0{,}8260$). Относительное отклонение по математическому ожиданию μ составило $0{,}29\,\%$, а по среднеквадратичному отклонению σ — $1{,}32\,\%$. Столь малая погрешность подтверждает статистическую адекватность разработанного генератора и свидетельствует о том, что сокращение объёма выборки до половины не привело к потере точности вос-

произведения законов распределения.

По данным системного лога, фактическое время работы успешных задач находилось в диапазоне от 2 до 360 секунд при среднем значении 180,56 с и стандартном отклонении 83,30 с, что соответствует аналитическим свойствам усечённого нормального распределения на интервале [2, 360] с.

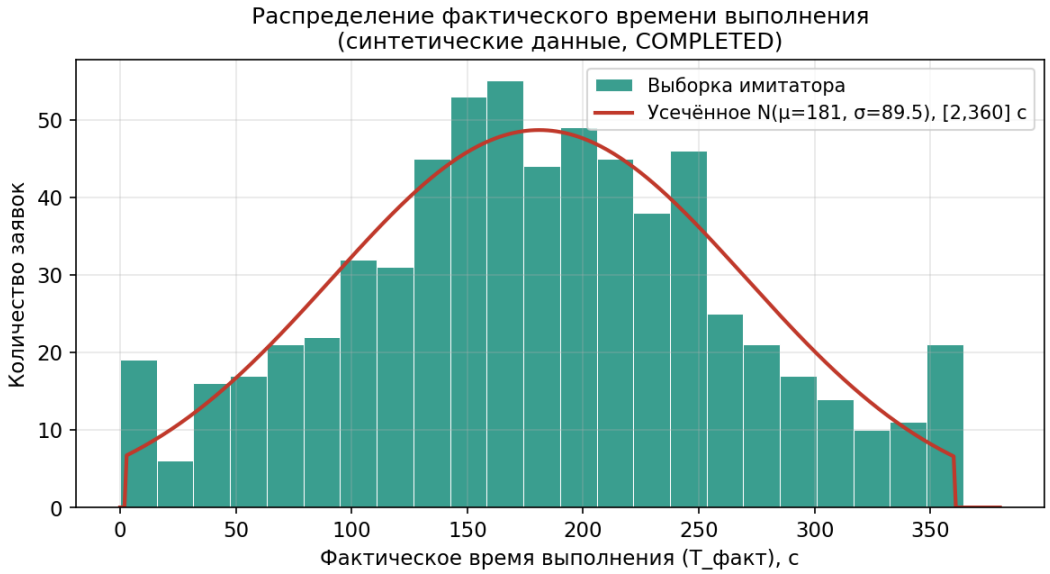


Рис. 5: Распределение фактического времени выполнения задач (синтетические данные, COMPLETED)

Исследование системных накладных расходов планировщика

Стандартные средства аудита SLURM ведут учёт времени выполнения задач дискретно, с точностью до целой секунды (ElapsedRaw [9]), что нивелирует микрозадержки операционной системы и самого планировщика. Для детального анализа накладных расходов диспетчеризации в рамках данной работы был задействован высокоточный аппаратный таймер Linux через вызов утилиты date +%s%3N внутри обёртки выполняемых задач.

Анализ накопленных данных файла overhead_log.csv позволил рассчитать абсолютную величину накладных расходов (системного оверхеда Δ_{overhead}) для каждой выполненной задачи как разность между полным временем работы сценария в миллисекундах и временем запрошенной полезной нагрузки sleep ($T_{\text{факт}} \cdot 1000$):

$$\Delta_{\text{overhead}} = \text{ActualElapsed_Ms} - (\text{SleepRequested_Sec} \cdot 1000). \qquad (3)$$

Статистический анализ по всем 716 выполненным задачам показал, что величина накладных расходов колеблется в диапазоне от 15 до 120 мс. Среднее значение системной задержки составило $\Delta_{\text{avg}} = 66{,}50$ мс при стандартном отклонении 29,92 мс. Данные задержки складываются из времени, затрачиваемого демоном slurmd на форк процесса, инициализацию окружения пользователя, запуск интерпретатора командной строки Bash и выполнение системных вызовов логирования. Экспериментально установлено, что при выбранном масштабе времени ($k = 30$, где задачи длятся десятки и сотни секунд) средний оверхед системы составляет не более 0,04 % от времени работы задачи, что позволяет пренебречь аппаратными задержками при анализе эффективности планирования.

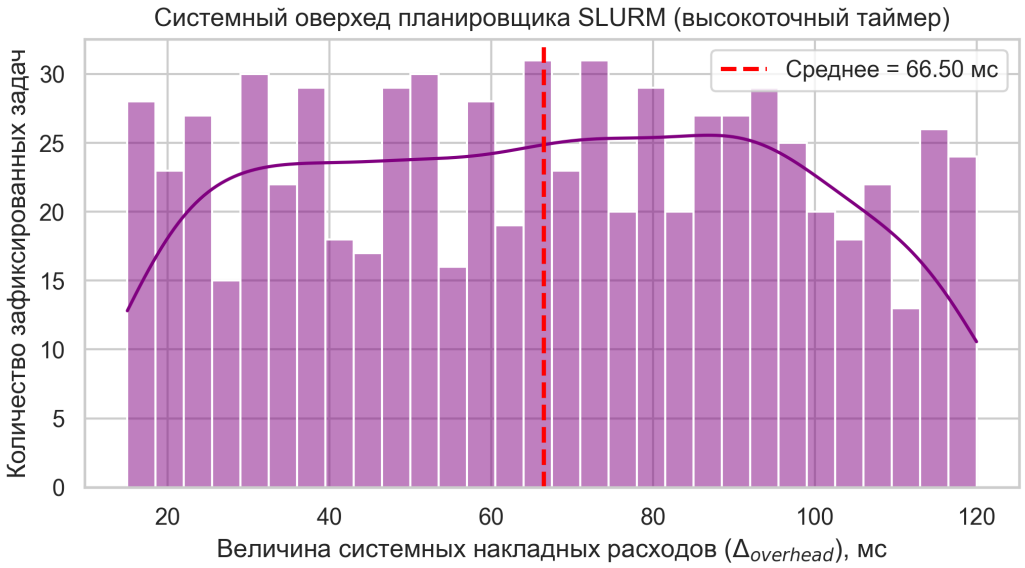


Рис. 6: Распределение системных накладных расходов (Δ_{overhead}) при диспетчеризации задач в SLURM, мс

Выводы по результатам экспериментов

Проведённый комплексный эксперимент показал, что разработанный имитатор пользовательских заявок на языке Python обеспечивает корректную и полностью автономную постановку заданий в очередь пакетного диспетчера SLURM.

Все сгенерированные заявки были успешно обработаны планировщиком, распределены по вычислительным узлам кластера и доведены до терминальных состояний (COMPLETED или TIMEOUT) в строгом соответствии с математической моделью. Эмпирические оценки параметров поль-

зовательской ошибки ($\hat{\mu} = 3{,}9295$, $\hat{\sigma} = 0{,}8151$) воспроизводят теоретические характеристики исходного датасета с минимальной погрешностью, что подтверждает применимость предложенного статистического подхода для моделирования реалистичной вычислительной нагрузки на кластерные системы в условиях отсутствия прямого доступа к промышленным суперкомпьютерным логам в реальном времени.

ЗАКЛЮЧЕНИЕ

В процессе выполнения НИР были изучены принципы организации виртуальной вычислительной среды, имитирующей ресурсы вычислительного кластера, а также технологии управления заданиями средствами системы SLURM на базе дистрибутива Linux Fedora Server в среде виртуализации Oracle VM VirtualBox. В результате работы были выполнены поставленные задачи:

1. Установлена и настроена среда виртуализации Oracle VM VirtualBox.
2. Создано пять виртуальных машин: один управляющий узел и четыре вычислительных узла кластера.
3. Установлен дистрибутив Linux Fedora Server (архитектура aarch64) на все виртуальные машины в режиме интерфейса командной строки.
4. Настроена виртуальная локальная сеть между узлами кластера с разрешением имён посредством файла /etc/hosts.
5. Настроено подключение по протоколу SSH с авторизацией по криптографическим ключам между управляющим и вычислительными узлами.
6. Установлена и настроена служба аутентификации munge, а также диспетчер заданий SLURM.
7. Выполнено клонирование настроенного вычислительного узла для создания оставшихся узлов кластера с последующей индивидуализацией их сетевых параметров.
8. Проведён анализ набора учётных данных acct_0921-0923 по заданным атрибутам варианта (UID 50430, JobName “run_bwa.sh”), вычислены границы диапазона фактического времени выполнения задач, а также параметры μ и σ логарифма ошибки оценки времени выполнения.
9. Разработан имитатор пользовательских заявок, генерирующий фактическое время выполнения задачи по нормальному распределению в вычисленном диапазоне и пользовательскую оценку времени выполнения по логнормальному распределению с вычисленными параметрами, с автоматической постановкой сформированных заявок в очередь диспетчера SLURM.
10. Реализован сбор и сохранение в файл статистики поставленных в оче-

редь заявок для последующего сравнительного анализа плановых и фактических характеристик их выполнения.

Таким образом, в ходе выполнения НИР развёрнута работоспособная виртуальная модель вычислительного кластера, на базе которой реализован и экспериментально проверен имитатор пользовательской нагрузки, статистические характеристики которого согласуются с параметрами, полученными в результате анализа реального набора учётных данных. Полученные результаты могут быть использованы в качестве основы для дальнейшего расширения функциональности диспетчера и более детального анализа эффективности алгоритмов планирования заданий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fedora Project Contributors. Fedora Linux Documentation. — 2024. — Дата обращения: 25.06.2026. https://docs.fedoraproject.org/.
2. Feitelson D. G. Workload Modeling for Computer Systems Performance Evaluation. — Cambridge : Cambridge University Press, 2015. — С. 564. — ISBN 978-1-107-07823-9.
3. Job Failures in High Performance Computing Systems: A Large-scale Empirical Study / S. Di [и др.] // Computers & Mathematics with Applications. — 2014. — Т. 68, № 10. — С. 1224—1236. — DOI: 10.1016/j.camwa.2014.08.010.
4. Kappes T., Hanzálek Z. User Estimates Inaccuracy Study in HPC Scheduler // Parallel Processing and Applied Mathematics. Т. 10778. — Cham : Springer, 2018. — С. 486—495. — (Lecture Notes in Computer Science). — DOI: 10.1007/978-3-319-91446-6_47.
5. Mu'alem A. W., Feitelson D. G. Utilization, Predictability, Workloads, and User Runtime Estimates in Scheduling the IBM SP2 with Backfilling // IEEE Transactions on Parallel and Distributed Systems. — 2001. — Т. 12, № 6. — С. 529—543. — DOI: 10.1109/71.932708.
6. Oracle Corporation. Oracle VM VirtualBox User Manual, Release 7.2. — 2025. — Дата обращения: 25.06.2026. https://www.virtualbox.org/manual/.
7. Python Software Foundation. Python 3 Documentation. — 2024. — Дата обращения: 25.06.2026. https://docs.python.org/3/.
8. SchedMD LLC. Slurm Workload Manager — Documentation. — 2024. — Дата обращения: 25.06.2026. https://slurm.schedmd.com/documentation.html.
9. SchedMD LLC. Slurm Workload Manager — sacct Command Reference. — 2024. — Дата обращения: 25.06.2026. https://slurm.schedmd.com/sacct.html.

10. SchedMD LLC. Slurm Workload Manager — sbatch Command Reference. — 2024. — Дата обращения: 25.06.2026. https://slurm.schedmd.com/sbatch.html.

11. Sherrill C., Planas V., Golden J. Mastering HPC Runtime Prediction: From Observing Patterns to a Methodological Approach : тех. отч. / National Renewable Energy Laboratory. — Golden, CO, 2023. — NREL/TP-2C00—86526. — URL: https://docs.nrel.gov/docs/fy23osti/86526.pdf.

12. Tsafrir D., Etsion Y., Feitelson D. G. Backfilling Using System-Generated Predictions Rather Than User Runtime Estimates // IEEE Transactions on Parallel and Distributed Systems. — 2007. — Т. 18, № 6. — С. 789—803. — DOI: 10.1109/TPDS.2007.70606.

13. Yoo A. B., Jette M. A., Grondona M. SLURM: Simple Linux Utility for Resource Management // Job Scheduling Strategies for Parallel Processing. Т. 2862. — Berlin, Heidelberg : Springer, 2003. — С. 44—60. — (Lecture Notes in Computer Science). — DOI: 10.1007/10968987_3.

14. Гмурман В. Теория вероятностей и математическая статистика. — 10-е изд. — Москва : Высшая школа, 2003. — С. 479.

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД

В приложении приведён полный исходный код программного имитатора пользовательских заявок (simulator.py), разработанного на языке Python 3. Программа выполняет статистический анализ набора учётных данных SLURM, генерирует синтетические заявки в соответствии с нормальным и логнормальным законами распределения и автоматически ставит их в очередь диспетчера посредством утилиты sbatch.

```
import os
import sys
import random
import math
import csv
import shutil
import subprocess
from datetime import datetime

UID = 50430
JOB_NAME = "run_bwa.sh"

K_SCALE = 30
T_MIN = 1
T_MAX = 360

MU    = 3.9411
SIGMA = 0.8260

TIMEOUT_RATE = 0.05

HOME_DIR  = "/home/vlad"
JOBS_DIR  = f"{HOME_DIR}/saved_jobs"
LOG_FILE  = f"{HOME_DIR}/simulation_stats.csv"


def generate_job():

    mean_t = (T_MIN + T_MAX) / 2
    std_t  = (T_MAX - T_MIN) / 4
    while True:
        t_fact = random.normalvariate(mean_t, std_t)
        if T_MIN <= t_fact <= T_MAX:
            break
    t_fact = max(2, round(t_fact))

    is_timeout = random.random() < TIMEOUT_RATE
```

```
38
39      if is_timeout:
40          if t_fact > 60:
41              max_minutes = (t_fact - 1) // 60
42              if max_minutes >= 1:
43                  limit_minutes = random.randint(1, max_minutes)
44              else:
45                  limit_minutes = 1
46              t_req = limit_minutes * 60
47          else:
48              is_timeout = False
49
50      if not is_timeout:
51          tau    = random.lognormvariate(MU, SIGMA)
52          t_req = round(t_fact * tau)
53          if t_req <= t_fact:
54              t_req = t_fact + random.randint(5, 15)
55
56      nodes = random.choice([1, 2, 3, 4])
57
58      return t_fact, t_req, nodes, is_timeout
59
60
61  def format_timelimit(t_req_sec):
62      mm = t_req_sec // 60
63      ss = t_req_sec % 60
64      return f"{mm:02d}:{ss:02d}"
65
66
67  def submit_to_slurm(job_id, t_fact, t_req, nodes):
68      timelimit_str = format_timelimit(t_req)
69
70      sbatch_content = f"""#!/bin/bash
71  #SBATCH --job-name={JOB_NAME}
72  #SBATCH --nodes={nodes}
73  #SBATCH --time={timelimit_str}
74  #SBATCH --output=/dev/null
75  #SBATCH --error=/dev/null
76
77  # Замер времени с точностью до миллисекунды для анализа накладных расходов
        SLURM
78  _START_MS=$(date +%s%3N)
79  sleep {t_fact}
80  _END_MS=$(date +%s%3N)
81  _ELAPSED_MS=$((_END_MS - _START_MS))
82
83  # Запись в лог накладных расходов: JobID, запрошенный sleep (с), реальное врем
        я (мс)
```

```
echo "${{SLURM_JOB_ID}},{t_fact},${{_ELAPSED_MS}}" >> {HOME_DIR}/overhead_log.csv
"""

    script_name = f"job_{job_id:03d}.sh"

    with open(script_name, "w") as f:
        f.write(sbatch_content)

    try:
        result = subprocess.run(
            ["sbatch", script_name],
            capture_output=True, text=True, check=True
        )
        slurm_job_id = result.stdout.strip().split()[-1]
        will_timeout = t_req < t_fact
        print(
            f"  Заявка {job_id:03d} | SlurmID: {slurm_job_id:>6} | "
            f"Лимит: {t_req:>4} с ({timelimit_str}) | "
            f"Факт: {t_fact:>4} с | "
            f"Узлов: {nodes} | "
            f"{'[TIMEOUT]' if will_timeout else ''}"
        )

        final_name = f"job_{job_id:03d}_slurm_{slurm_job_id}.sh"
        shutil.move(script_name, os.path.join(JOBS_DIR, final_name))
        return slurm_job_id

    except Exception as e:
        print(f"  Ошибка отправки заявки {job_id:03d} в SLURM: {e}")
        if os.path.exists(script_name):
            shutil.move(script_name, os.path.join(JOBS_DIR, f"job_{job_id:03d}_ERROR.sh"))
        return "ERROR"


def main():
    if len(sys.argv) < 2:
        print("Использование: python3 simulator.py <количество_заявок>")
        print("  Количество заявок: целое число от 1 до 999")
        sys.exit(1)

    try:
        num_jobs = int(sys.argv[1])
        if not (1 <= num_jobs <= 999):
            raise ValueError
    except ValueError:
        print("Ошибка: количество заявок должно быть целым числом от 1 до 999.
```

```
    ")
130         sys.exit(1)
131
132     if os.path.exists(JOBS_DIR):
133         shutil.rmtree(JOBS_DIR)
134     os.makedirs(JOBS_DIR, exist_ok=True)
135
136     overhead_log = f"{HOME_DIR}/overhead_log.csv"
137     if not os.path.exists(overhead_log):
138         with open(overhead_log, "w") as f:
139             f.write("SlurmJobID,SleepRequested_Sec,ActualElapsed_Ms\n")
140
141     print(f"=== Имитатор заявок: запуск генерации {num_jobs} заявок ===")
142     print(f"    Параметры модели: T_MIN={T_MIN} c, T_MAX={T_MAX} c, k={K_SCALE
    }")
143     print()
144
145     with open(LOG_FILE, mode="w", newline="") as f:
146         writer = csv.writer(f)
147         writer.writerow([
148             "Timestamp", "SimID", "SlurmJobID", "Nodes",
149             "PlannedTime_Sec", "ActualTime_Sec", "Tau", "IsTimeout"
150         ])
151
152         for i in range(1, num_jobs + 1):
153             t_fact, t_req, nodes, is_timeout = generate_job()
154             tau = round(t_req / t_fact, 4)
155             slurm_id = submit_to_slurm(i, t_fact, t_req, nodes)
156             writer.writerow([
157                 datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
158                 i, slurm_id, nodes,
159                 t_req, t_fact, tau, int(is_timeout)
160             ])
161
162     print()
163     print(f"=== Генерация завершена. Статистика сохранена в {LOG_FILE} ===")
164     print(f"    Сценарии задач сохранены в: {JOBS_DIR}")
165
166
167 if __name__ == "__main__":
168     main()
```

Листинг 9: Имитатор пользовательских заявок simulator.py